

[Sudip Chakrabarty \(https://learnopencv.com/author/sudip/\)](#)

on April 21, 2026

YOLO26 Keypoint Estimation: Real-Time Pose Estimation with Ultralytics

Learn how to use YOLO26-pose with Python for real-time keypoint estimation on images and videos, understand its RLE-based architecture, and explore its reported benchmarks on COCO-17.

[Computer Vision \(https://learnopencv.com/category/computer-vision/\)](#)[YOLO \(https://learnopencv.com/category/yolo/\)](#)

Human pose estimation has become a cornerstone of modern computer vision, powering applications from fitness tracking apps and sports analytics to gesture-based interfaces and medical rehabilitation. At its core, **keypoint estimation** is the task of detecting specific anatomical landmarks on the human body, the nose, shoulders, elbows, wrists, hips, knees, and ankles, and connecting them into a skeleton that represents the person's pose.

In September 2025, Ultralytics announced **YOLO26**, the next-generation YOLO model optimized for edge computing, robotics, and mobile AI. Among its specialized task heads, **YOLO26-pose** brings several architectural innovations to keypoint estimation: **Residual Log-Likelihood Estimation (RLE)** for more accurate keypoint localization, **end-to-end**

NMS-free inference for simpler deployment, and the **MuSGD optimizer** for more stable training dynamics. In this guide, we walk through the theory, architecture, benchmarks, and a hands-on implementation of YOLO26 keypoint estimation on images and videos.

Table of Contents

Table of Contents

1. Introduction to Keypoint Estimation
2. YOLO26 Pose Architecture Overview
3. Understanding COCO Keypoints
4. Benchmarks and Performance Metrics
5. Getting Started with YOLO26 Keypoint Estimation
6. Inference on Video Data
7. Qualitative Results
8. Real-World Use Cases
9. Conclusion and Future Outlook
10. References and Resources

1. Introduction to Keypoint Estimation

For years, pose estimation has been dominated by two broad families of approaches: **top-down methods** (HRNet, SimpleBaseline) that first detect people and then estimate keypoints on cropped regions, and **bottom-up methods** (OpenPose, HigherHRNet) that detect all keypoints first and then group them into individual poses. Both families achieve strong accuracy but come with trade-offs: top-down methods scale poorly with crowd size, while bottom-up methods struggle with grouping in dense scenes.

The YOLO family took a radically different approach. By predicting bounding boxes and keypoints **simultaneously in a single forward pass**, YOLO-pose models eliminate the separate detection and grouping stages. This single-stage paradigm delivers real-time speed without sacrificing the end-to-end simplicity that has made YOLO popular. YOLO26 builds on this foundation with targeted improvements. Most notably, **Residual Log-Likelihood Estimation (RLE)** for more precise keypoint coordinates, and the removal of Non-Maximum Suppression (NMS) from the inference pipeline, giving you predictable latency on edge devices. For a closer look at how YOLO26 achieves this, read [our deep dive on NMS-free inference](https://learnopencv.com/yolo26-nms-free-inference/) (<https://learnopencv.com/yolo26-nms-free-inference/>).

2. YOLO26 Pose Architecture Overview

YOLO26-pose consists of three main components: a convolutional backbone for feature extraction, a multi-scale feature-fusion neck, and a dedicated pose head that predicts keypoints alongside object detections.

2.1 Backbone

The backbone is a feature extraction network that processes the input image and generates multi-scale feature maps. These feature maps capture both low-level details (edges, textures) and high-level semantic information needed to reason about human body parts. YOLO26 delivers up to **43% faster CPU inference** compared to its predecessors, making it suitable even for mobile and edge deployments. For a hands-on walkthrough, see [our guide on YOLO26 real-time deployment](https://learnopencv.com/yolov26-real-time-deployment/) (<https://learnopencv.com/yolov26-real-time-deployment/>).

2.2 Neck (Feature Fusion)

The neck module fuses features from different scales of the backbone. This multi-scale fusion is critical for detecting persons of different sizes, from small, distant figures to large, close-up subjects, without losing fine-grained keypoint accuracy. YOLO26 also removes Distribution Focal Loss (DFL), reducing complexity while expanding hardware compatibility for edge accelerators.

2.3 Pose Head with RLE

A key innovation in YOLO26 is the integration of **Residual Log-Likelihood Estimation (RLE)** in the pose head. RLE learns the underlying keypoint-uncertainty distribution directly from data rather than assuming a fixed prior, leading to more accurate keypoint localization, particularly on unusual poses and partial occlusions.

For each detected person, the head predicts:

- Bounding box coordinates (x, y, w, h)
- Object confidence score
- Class probability (person)
- **17 × 3 keypoint values:** (x, y, confidence) for each of the 17 COCO keypoints

2.4 Training Improvements

YOLO26 introduces the **MuSGD optimizer**, a hybrid of SGD and the Muon algorithm, for more stable training dynamics. Combined with **ProgLoss + STAL** improvements, it achieves better accuracy particularly for detecting small objects, which is critical for keypoint localization where body joints can span only a few pixels.

3. Understanding COCO Keypoints

YOLO26-pose is pretrained on the **COCO Keypoints dataset**, which defines **17 keypoints** for human pose estimation. These keypoints cover the major joints and facial landmarks:

COCO-17 Keypoint Mapping

DATASET

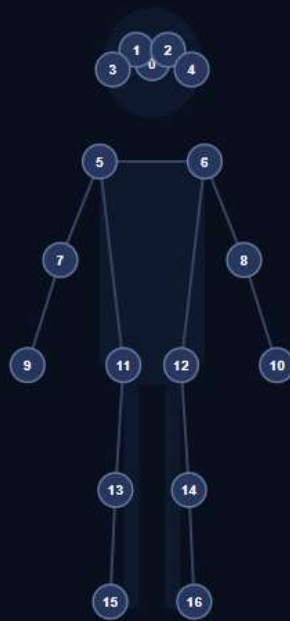
COCO Keypoints

TOTAL POINTS

17

CATEGORIES

Face, Arms, Torso, Legs

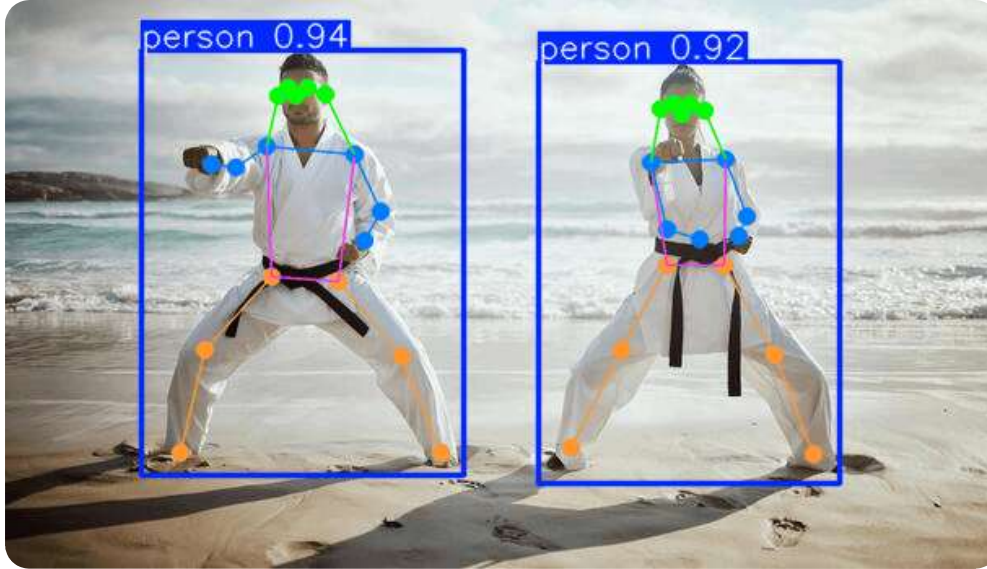


17 keypoints covering facial landmarks and major body joints — the standard mapping used by YOLO26-pose and COCO-format models.

(<https://learnopencv.com/wp-content/uploads/2026/04/Keypoints.png>)

These keypoints are connected by skeleton edges: face edges linking the nose to the eyes and ears, upper-body edges along shoulder-elbow-wrist chains, torso edges between the shoulders and hips, and lower-body edges along hip-knee-ankle chains. Together, they form the human stick-figure overlay commonly seen in pose estimation visualizations.

Below is an example of YOLO26-pose running on a karate scene, showing the full 17-keypoint skeleton overlaid on each detected person:



4. Benchmarks and Performance Metrics

Ultralytics publishes five YOLO26-pose variants, scaling from a tiny 2.9M-parameter Nano model to a 57.6M-parameter X-Large model. The table below summarizes their performance on the COCO Keypoints val2017 dataset:

Model	Params (M)	FLOPs (B)	mAP ₅₀₋₉₅ (Pose)	mAP ₅₀ (Pose)	CPU ONNX (ms)	T4 TensorRT10 (ms)
YOLO26n- pose	2.9	7.5	57.2	83.3	40.3 ± 0.5	1.8 ± 0.0
YOLO26s- pose	10.4	23.9	63.0	86.6	85.3 ± 0.9	2.7 ± 0.0
YOLO26m- pose	21.5	73.1	68.8	89.6	218.0 ± 1.5	5.0 ± 0.1
YOLO26l- pose	25.9	91.3	70.4	90.5	275.4 ± 2.4	6.5 ± 0.1

Model	Params (M)	FLOPs (B)	mAP ₅₀₋₉₅ (Pose)	mAP ₅₀ (Pose)	CPU ONNX (ms)	T4 TensorRT10 (ms)
YOLO26x- pose	57.6	201.7	71.6	91.6	565.4 ± 3.0	12.2 ± 0.2

All benchmarks measured at 640 × 640 input resolution on the COCO Keypoints val2017 dataset. Speed reported with mean ± standard deviation across multiple runs.

The table highlights the flexible scaling curve of YOLO26-pose. The Nano model achieves a remarkable **1.8 ms** inference latency on a T4 GPU, ideal for real-time webcam applications, while the X-Large variant pushes accuracy to **71.6 mAP₅₀₋₉₅** for offline analysis pipelines. For most tutorials and Colab notebooks, the Nano or Small variants strike the best balance between speed and accuracy.

5. Getting Started with YOLO26 Keypoint Estimation

Ultralytics makes it easy to get started with YOLO26. The library handles model download, preprocessing, inference, and visualization in a few lines of Python.

5.1 Installation

Install Ultralytics with a single pip command:

```
1 | pip install ultralytics
```

This pulls in PyTorch, OpenCV, and all required dependencies.

5.2 Running Keypoint Estimation on an Image

Load the pretrained YOLO26-pose model and run inference on a single image:

```

1  from ultralytics import YOLO
2  import cv2
3
4  # Load the YOLO26 pose model (auto-downloads on first run)
5  model = YOLO("yolo26n-pose.pt")
6
7  def run_keypoint_on_image(image_path, output_path="output_image.jpg", conf=0.5):
8      """Run YOLO26 keypoint estimation on a single image."""
9      results = model(image_path, conf=conf)
10
11     for result in results:
12         # Draw skeleton + keypoints on image
13         annotated_frame = result.plot()
14         cv2.imwrite(output_path, annotated_frame)
15
16         # Print keypoint info
17         if result.keypoints is not None:
18             keypoints = result.keypoints.xy # shape: (N_persons, 17, 2)
19             print(f"Detected {len(keypoints)} person(s)")
20             print(f"Keypoints shape: {keypoints.shape}")
21
22     return results
23
24 if __name__ == "__main__":
25     run_keypoint_on_image("yoga.png", output_path="output_image.jpg")

```

Running this on the yoga test image gives the following result. This is also a good place to see where the model struggles. Challenging yoga poses, with inverted limbs push single-stage pose estimators to their limits:



Looking closely at the output, you can see that YOLO26-pose does not recover every keypoint here. A few joints are placed incorrectly and some are missed entirely, which distorts the skeleton lines that connect them. This kind of failure mode is typical for challenging yoga, gymnastics, and acrobatic poses where limbs fold across the body and self-occlusion is severe. It is a known limitation of single-stage pose estimators, and something to keep in mind when applying YOLO26-pose to unusual body configurations. In the sections below we look at a wider set of inputs to see where the model holds up well and where it still struggles.

5.3 Accessing Raw Keypoint Data

Beyond the default visualization, `result.keypoints` exposes the raw coordinates and confidence scores so you can build custom analytics: joint-angle computation, rep counting, form checks, gesture recognition, or feeding a downstream model:

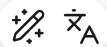
```

NAMES = [
    'Left Eye', 'Right Eye', 'Left Ear', 'Right Ear',
    'Shoulder', 'Right Shoulder', 'Left Elbow', 'Right Elbow',
    'Wrist', 'Right Wrist', 'Left Hip', 'Right Hip',
    'Knee', 'Right Knee', 'Left Ankle', 'Right Ankle',

model("karate.png", conf=0.5)

for result in results:
    if result.keypoints is None:
        continue
    for person_idx, kps in enumerate(result.keypoints.xy):
        print(f"\n--- Person {person_idx + 1} ---")
        for i, (x, y) in enumerate(kps):
            if x > 0 and y > 0:
                conf_score = result.keypoints.conf[person_idx][i].item()
                print(f"  {KEYPOINT_NAMES[i]:20s}: ({x:.1f}, {y:.1f})  conf={conf_score:.3f}")

```



The `result.keypoints` tensor has shape $(N, 17, 2)$ where N is the number of detected persons. The companion `result.keypoints.conf` tensor holds per-keypoint confidence scores.

5.4 Multi-Person Detection

As a single-stage detector, YOLO26-pose handles multiple persons in one forward pass, no grouping heuristics required. Each person gets its own bounding box and 17-keypoint skeleton:



YOLO26-pose tracks multiple people simultaneously, maintaining individual skeletons even when people overlap or stand close together.

6. Inference on Video Data

Static image inference is useful for photo analysis, but the true value of pose estimation emerges on video, for form checking, motion analysis, and real-time feedback. Below is a frame-by-frame video processing pipeline that annotates every frame with the YOLO26-pose skeleton and writes out an annotated output video:

```

1  from ultralytics import YOLO
2  import cv2
3
4  # Load the YOLO26 pose model
5  model = YOLO("yolo26n-pose.pt")
6
7  def run_keypoint_on_video(video_path, output_path="output_video.mp4", conf=0.5):
8      """Run YOLO26 keypoint estimation on a video file frame by frame."""
9      cap = cv2.VideoCapture(video_path)
10     if not cap.isOpened():
11         print(f"Error: Could not open video '{video_path}'")
12         return
13
14     fps = int(cap.get(cv2.CAP_PROP_FPS))
15     width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
16     height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
17     total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
18
19     print(f"Processing : {video_path}")
20     print(f"Resolution : {width}x{height} @ {fps}fps | {total} total frames")
21
22     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
23     out = cv2.VideoWriter(output_path, fourcc, fps, (width, height))
24
25     frame_count = 0
26     while cap.isOpened():
27         ret, frame = cap.read()
28         if not ret:
29             break
30
31         results = model(frame, conf=conf, verbose=False)
32         annotated_frame = results[0].plot()
33         out.write(annotated_frame)
34
35         frame_count += 1
36         if frame_count % 30 == 0:
37             print(f"  Processed {frame_count}/{total} frames...")
38

```

```

39     cap.release()
40     out.release()
41     print(f"Done! Output saved to: {output_path}")
42
43     if __name__ == "__main__":
44         run_keypoint_on_video("dance.mp4", output_path="output_dance.mp4")

```

On fast, rhythm-based movement such as dance, YOLO26-pose tracks the skeleton through wide arm swings and rapid orientation changes without dropping detections:



YOLO26-pose frame-by-frame inference on a dance clip. The model maintains skeleton continuity through fast arm swings and body rotations.

7. Qualitative Results

To get a feel for YOLO26-pose in the wild, we evaluated it on several categories of real-world inputs. The following results use the Nano variant (`yo1o26n-pose.pt`), which is well-suited to Colab's T4 GPU and to edge deployment.

7.1 Gym Workouts: Exercise Form Analysis

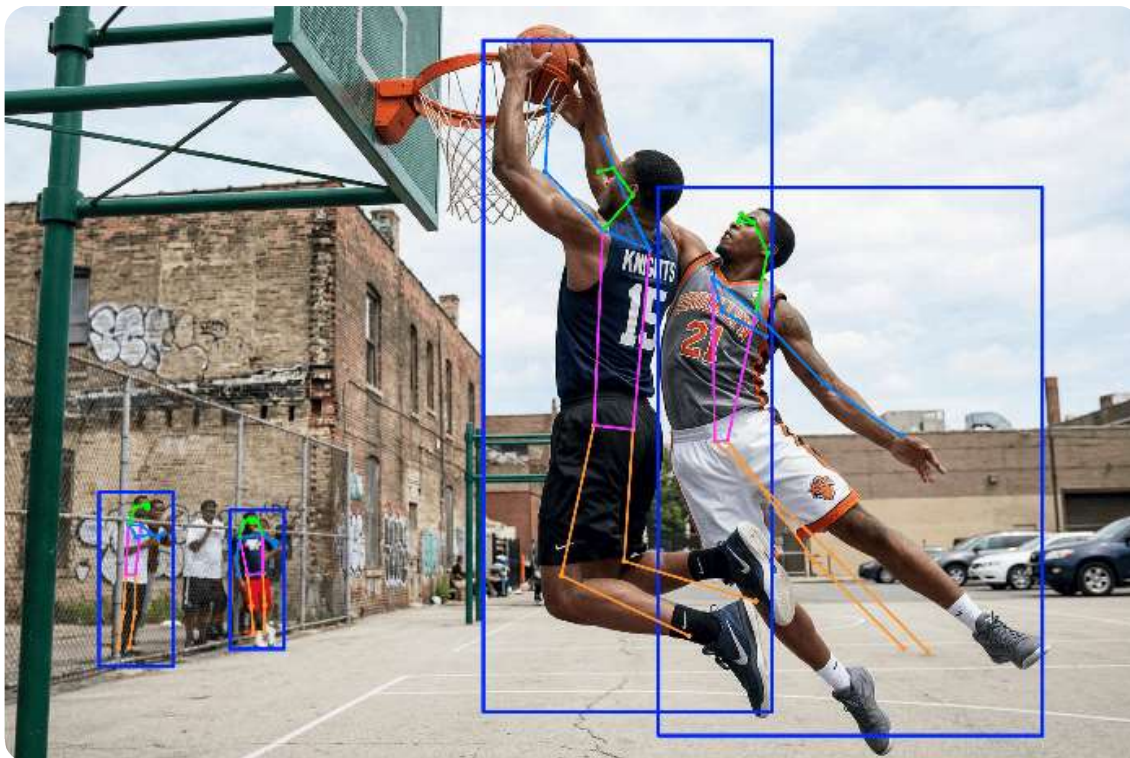
Gym workouts involve repetitive, well-defined movements like squats, presses, and lifts. Pose estimation here has immediate practical value: left/right symmetry analysis, range-of-motion tracking, rep counting, and form feedback:



YOLO26-pose on a gym scene. The skeleton cleanly captures the stance and posture, enabling downstream form checks for personal trainers and fitness apps.

7.2 Sports Action: Mid-Air and Occluded Poses

Sports action shots are some of the hardest inputs for pose estimation: fast, non-standard poses with overlapping bodies and partial occlusion from equipment like the ball and net. YOLO26's RLE-based head helps maintain localization accuracy even on these challenging frames:



Sports action test. YOLO26-pose captures both main athletes, but the defender's lower-body skeleton shows some tangled lines where the two bodies overlap, and one figure standing behind the fence in the background is missed

Zooming in, it shows where the model struggles. When two bodies overlap this tightly, single-stage pose estimators can blur the boundary between people and some keypoint connections get miswired across the two skeletons. Small or partially hidden figures in the background, like the person behind the fence here, can also fall below the detection confidence threshold and get missed entirely. Both are expected limitations, and they are useful to keep in mind when applying YOLO26-pose to crowded real-world scenes.

7.3 Dance: Dynamic Full-Body Motion

As shown in the video in Section 6, dance captures large, fluid body motions: wide arm arcs, rapid rotations, and full-body coordination. This is where frame-by-frame tracking quality becomes critical, and where the single-stage speed of YOLO26 truly shines for real-time applications.

7.4 Jump: Tracking Through Fast, Mid-Air Transitions

This short clip shows the model handling a full jump sequence, from the crouch to takeoff and landing. YOLO26-pose tracks the skeleton reliably through the upright frames and picks up the athlete again on landing. During the brief inverted moment, when the body is fully upside down, detection drops because the model has seen very few such orientations during COCO pretraining. For real applications, a simple temporal smoother or interpolation step is usually enough to bridge those short gaps.



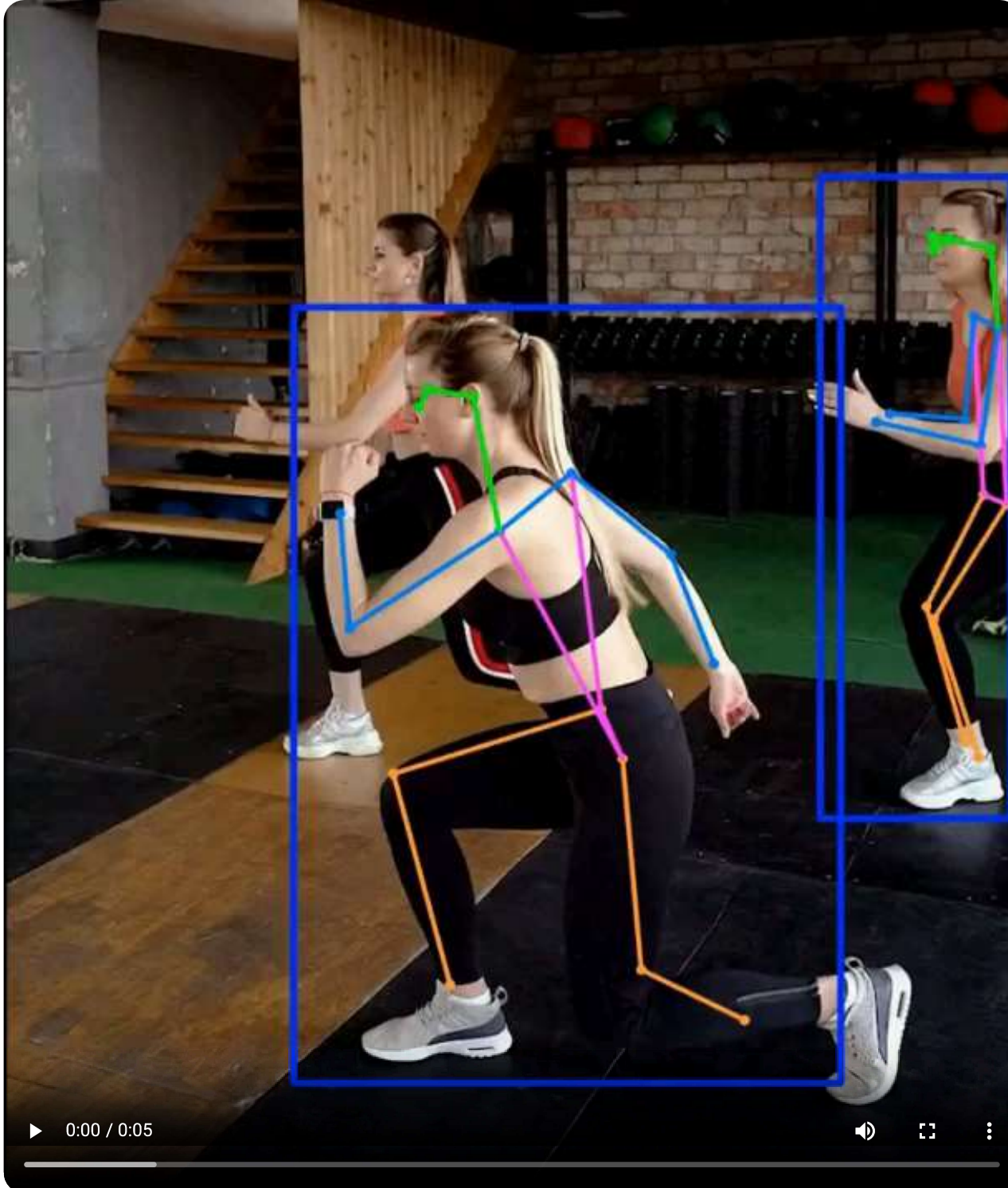
7.5 Parkour: Multiple Athletes in a Complex Scene

This rooftop parkour clip is a much harder input than any of the static image examples: a moving camera, two athletes in the frame, a wide depth range, and mid-jump body configurations. Even so, YOLO26-pose holds up well. Both skeletons stay attached to the right people through most of the clip, and the joint positions look stable even while the athletes are airborne. This is the kind of scene where the single-stage design really pays off, since there is no separate per-person cropping step that could lose identity between frames.



7.6 Yoga: Multi-Person Tracking With Bounding Boxes

This yoga clip also shows the detection bounding boxes around each person, along with the skeleton overlay. Those blue boxes come from the Ultralytics default visualization and make it easy to see which keypoints belong to which detected person. The model tracks both practitioners cleanly across the standing and balancing poses, and the skeletons stay attached to the right body even when the two come close together in the frame.



Related Post

Vision Banana: How Image Generators Are Becoming Powerful Vision Models
(<https://learnopencv.com/vision-banana-explained/>)

Vision Banana turns Nano Banana Pro into a powerful vision model for segmentation, depth estimation,



8. Real-World Use Cases

Keypoint estimation has a wide range of practical applications across industries:



(https://learnopencv.com/wp-content/uploads/2026/04/Use_case.jpg)

- **Sports Analytics:** Track athlete movements, measure joint angles, and provide data-driven coaching feedback in sports like cricket, baseball, golf, and swimming.
- **Fitness and Rehabilitation:** Build AI-powered fitness apps that count reps, check exercise form, and provide real-time corrective feedback. In physical therapy, track patient recovery by measuring range of motion over time.
- **Gesture Recognition and HCI:** Enable gesture-based interfaces for gaming, AR/VR, and accessibility applications. Pose data can feed gesture classifiers that recognize sign language, dance moves, or custom commands.
- **Surveillance and Safety:** Detect falls, unusual postures, or unsafe behaviors in workplaces and elderly care settings.
- **Animation and Motion Capture:** Use keypoint data as a lightweight motion capture solution for animating 3D characters, a cost-effective alternative to marker-based

RF-DETR Segmentation: Real-Time Detection & Instance Segmentation Guide
(<https://learnopencv.com/rf-detr-segmentation-real-time-detection-instance-segmentation-guide/>)

Learn how to use RF-DETR-Seg with Python for image and video inference, understand the architecture

YOLO26 Instance Segmentation: Pixel-Perfect AI at Real-Time Speed
(<https://learnopencv.com/yolc-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

Build a complete pipeline for YOLO26 instance segmentation, from image and video inference to custom

Table of Contents

Table of Contents

1. Introduction to Keypoint Estimation
2. YOLO26 Pose Architecture Overview
3. Understanding COCO Keypoints
4. Benchmarks and Performance Metrics
5. Getting Started with YOLO26 Keypoint Esti...
6. Inference on Video Data
7. Qualitative Results
8. Real-World Use Cases

100%



9. Conclusion and Future Outlook

YOLO26 marks a significant step forward for real-time keypoint estimation. By integrating **Residual Log-Likelihood Estimation (RLE)** into the pose head, the model delivers more accurate keypoint localization than its YOLO predecessors. By removing NMS from the inference pipeline, it achieves **end-to-end predictable latency**. This is an important property for edge deployment.

For developers and researchers, the practical impact is clear: with

`pip install ultralytics` and a few lines of Python, you can deploy a production-ready pose estimator that runs at **1.8 ms per frame** on a T4 GPU and scales smoothly to 57M-parameter configurations when you need maximum accuracy. Whether you are building a fitness coaching app, a sports analytics platform, an interactive installation, or a custom keypoint task beyond humans, YOLO26-pose provides a powerful and accessible foundation.

Looking ahead, we expect the YOLO-pose ecosystem to continue evolving along three axes: better temporal tracking for video (leveraging optical flow and transformer-based memory), richer keypoint schemas (WholeBody, hand keypoints, 3D poses), and tighter integration with edge accelerators (TensorRT, CoreML, TFLite) for on-device real-time applications.

10. References and Resources

- [Ultralytics YOLO26 Documentation](https://docs.ultralytics.com/models/yolo26/) (<https://docs.ultralytics.com/models/yolo26/>)
- [Ultralytics Pose Estimation Documentation](https://docs.ultralytics.com/tasks/pose/) (<https://docs.ultralytics.com/tasks/pose/>)
- [COCO Keypoint Detection Task](https://cocodataset.org/#keypoints-2020) (<https://cocodataset.org/#keypoints-2020>)
- [Residual Log-Likelihood Estimation \(RLE\) Paper](https://arxiv.org/abs/2107.11291) (<https://arxiv.org/abs/2107.11291>)
- [OpenPose: Realtime Multi-Person 2D Pose Estimation](https://arxiv.org/abs/1812.08008) (<https://arxiv.org/abs/1812.08008>)
- [LearnOpenCV Blog](https://learnopencv.com/) (<https://learnopencv.com/>)

Was This Article Helpful?



- [Related Article: YOLO26 – An Analysis of NMS-Free End-to-End Framework for Real-Time Object Detection](https://www.researchgate.net/publication/399931587_YOLO26_An_Analysis_of_NMS-Free_End-to-End_Framework_for_Real-Time_Object_Detection)
(https://www.researchgate.net/publication/399931587_YOLO26_An_Analysis_of_NMS-Free_End_to_End_Framework_for_Real-Time_Object_Detection)

Frequently Asked Questions

Was This Article Helpful?



Subscribe & Download Code

If you liked this article and would like to download code (C++ and Python) and example images used in this post, please [click here](#). Alternately, sign up to receive a free [Computer Vision Resource](#) Guide. In our newsletter, we share OpenCV tutorials and examples written in C++/Python, and Computer Vision and Machine Learning algorithms and news.

 [Download Example Code](#)

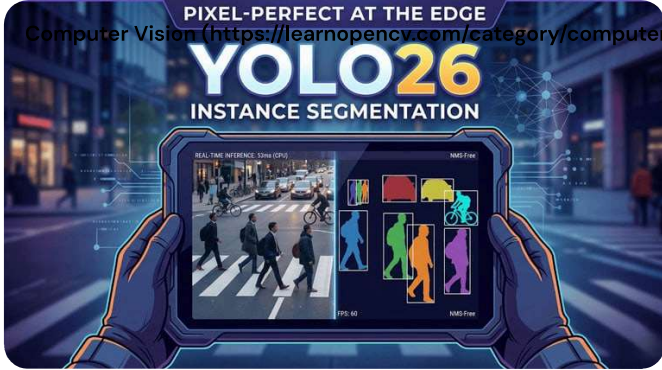


(<https://learnopencv.com/rf-detr-seg/> | learnopencv.com/vision-banana-explained/)

Load Comments

Read Next





(<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

Sudip Chakrabarty
(<https://learnopencv.com/author/sudip/>)

on March 26, 2026

YOLO26 Instance Segmentation: Pixel-Perfect AI at Real-Time Speed
(<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

Build a complete pipeline for YOLO26 instance segmentation, from image and video inference to custom dataset training and edge deployment.



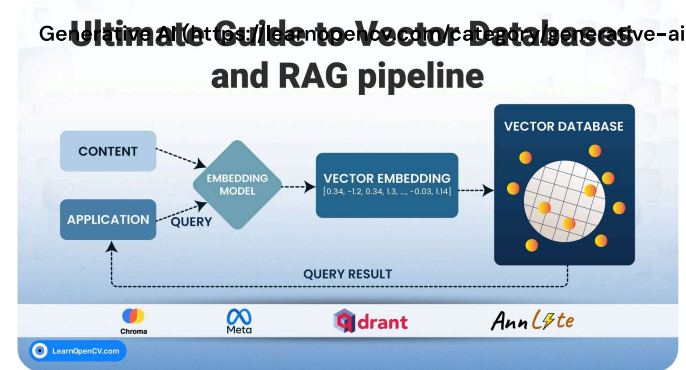
(<https://learnopencv.com/yolo26-nms-free-inference/>)

Sudip Chakrabarty
(<https://learnopencv.com/author/sudip/>)

on February 10, 2026

Breaking the Bottleneck: Achieving Native NMS-Free Inference with YOLO26
(<https://learnopencv.com/yolo26-nms-free-inference/>)

YOLO26 introduces a paradigm shift with native NMS-free inference. Discover how its One-to-One label assignment eliminates post-processing overhead for stable,...



(<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

Kukil (<https://learnopencv.com/author/kukil/>)

on November 11, 2025

The Ultimate Guide to Vector DB and RAG Pipeline (<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

Processing long documents with VLMs or LLMs poses a fundamental challenge: input size exceeds context limits. Even with GPUs, as...

Subscribe to our Newsletter

Subscribe to our email newsletter to get the latest posts delivered right to your email.



About LearnOpenCV

Empowering innovation through education, LearnOpenCV provides in-depth tutorials, code, and guides in AI, Computer Vision, and Deep Learning. Led by Dr. Satya Mallick, we're dedicated to nurturing a community keen on technology breakthroughs.

[\(https://learnopencv.com/about/\)](https://learnopencv.com/about/)

FREE Courses

- VLM Bootcamp
- PyTorch Bootcamp
- TensorFlow & Keras Bootcamp
- OpenCV Bootcamp
- Python for Beginners

Categories

- Deep Learning
- Object Detection
- Image Classification
- YOLO
- Image Processing
- Image Segmentation

Getting Started

- Installation
- PyTorch
- Getting Started with OpenCV
- Keras & Tensorflow

Big Vision

- AI Consulting
- Case Studies
- Get in Touch